USER GUIDE + SCRIPT REFERENCE

NWB Workbench

From converted NWB files to inspection, processing, tracking, spike sorting and analysis.

Read this guide if...	What you will learn
Your data are already in NWB	Create a project, index recordings and inspect their contents.
You want a repeatable workflow	Choose files and trials, run tools, compare conditions and retain results.
You want to add your own method	Write Processing, Analysis and spike-preprocessing scripts using the actual application interfaces.

Documentation revision 27 | 02 November 2026

Expanded and revised from the supplied v25 guide. Verified against the application source available for this revision, including recent DeepLabCut and trajectory-analysis changes. Interface details may differ in an older installed build.

Examples use generic recordings and illustrative data. Optional workflows are not required for every project. Captures show real Qt widgets; demo entries do not represent completed scientific analyses.

Contents

Choose your route

1. Start with a project 3

2. Files and filters 6

3. Inspect your data 8

4. Events and trials 11

5. Processing 13

6. DeepLabCut 15

7. Spike sorting 24

8. Analyses and conditions 30

9. Results and the Plot GUI 34

10. Write a processing script 37

11. Write an analysis script 45

12. Practical checks and troubleshooting 52

First use: chapters 1-4. Derive NWB data: chapter 5. Track video: chapter 6. Sort extracellular spikes: chapter 7. Compare conditions: chapters 8-9. Write your own methods: chapters 10-11.

1. Start with a project

What Workbench organizes

An NWB file is a recording container. A Workbench project is a working directory that indexes one or many NWB files and stores your scripts, selections, jobs and results. Creating a project does not convert your recordings again and does not combine all recordings into one NWB file.

Before your first run

• Have a working Workbench Python environment or a complete Windows application bundle. Keep an executable with its matching _internal directory.

• Know where the NWB files and any externally referenced videos are stored. Workbench cannot display a video whose external path is unavailable.

• Check that your conversion preserved timestamps, units, object names and subject/session metadata. A readable file can still contain scientifically incorrect timing.

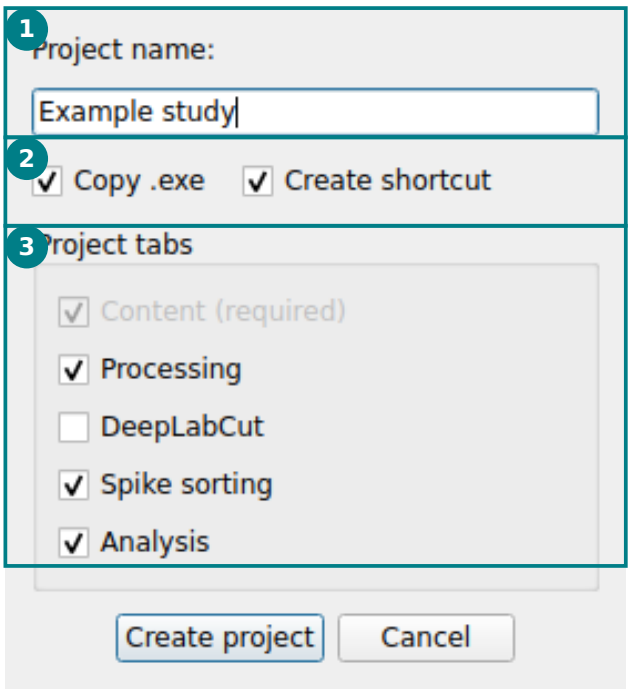
• Choose a stable project folder. Keep source recordings accessible at stable paths; a copied application is not a copy of the dataset.

Choose only the workflows you need

Workflow	Purpose	Where results go
Content	Inspect data and timing	Display or exported plots
Processing	Create derived NWB objects	Usually processing/<module> in NWB
DeepLabCut	Label, train and predict body-part positions	DLC project; explicit NWB export
Spike sorting	Detect and curate extracellular units	Sorting project; explicit NWB export
Analysis	Compare selected observations	Analysis result folders and figures

Workbench supports several NWB data types, but it is not a universal viewer or automatic analysis for every NWB extension. A type must be supported by the relevant viewer, extractor or selected script. The same project can contain different modalities; files need not all contain the same objects.

Create and reopen



Current project-creation dialog. 1: name. 2: copied Windows app and shortcut. 3: enabled workflow tabs.

1. Open the Workbench launcher and choose Create project. Select the parent directory and enter a project name.

2. Choose the tabs needed for the study. Content is required; optional workflows can be omitted. The selection is stored in nwb_workbench_project.json.

3. Copy .exe copies an existing prebuilt Windows bundle into the project. It does not compile a fresh application. Create shortcut depends on that option. For a Python-only project, clear Copy .exe.

4. Create the project. Once it opens, add recordings through the Files panel. The inventory is populated by scanning, not by project naming.

To return later, use Open project and select the project root, its Desktop shortcut, its copied executable, or launch_NWB_Workbench.py in the configured Python environment. Reopening normally loads the saved inventory instead of rescanning all recordings.

If a project executable opens directly and you need the launcher, start it with --launcher. When enabling a previously omitted tab through project metadata, close and reopen Workbench. Keep the enabled-tab identifiers consistent with the installed application.

Files and folders to keep

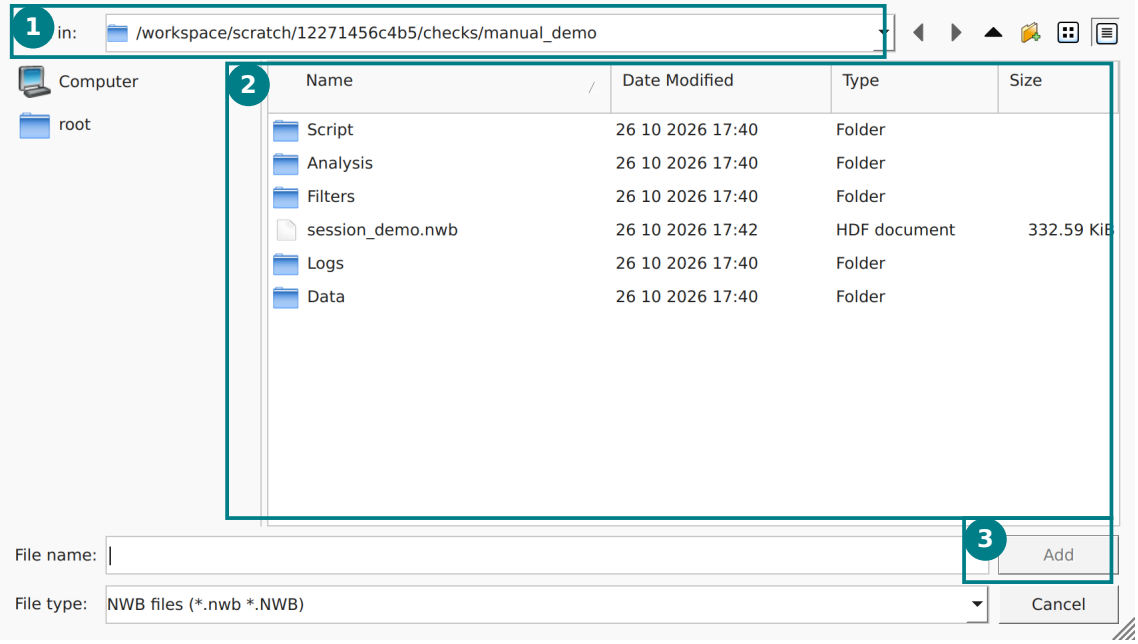
Project location	Purpose
nwb_workbench_project.json	Project identity, settings and enabled tabs.
Data/nwb_inventory.xlsx	Index of file metadata, indexed content and trial-filter information.
Filters/	Saved condition/filter definitions.
Script/Processing/<Category>/	Processing tools specific to this project.
Script/Analysis/<Category>/	Analysis scripts specific to this project.
Script/Spike_sorting/Preprocessing/	Project-specific SpikeInterface preprocessing.
Script/Utilities/	Helpers; not automatically an Analysis or Processing tool.
Analysis/	Saved figures, numerical outputs and analysis logs.
Data/Preprocessed_filtered_data/	Reusable extracted/filtered analysis data and its inventory.
Data/Spike_sorting/	Sorting recipes, runs, analyzers and curation outputs.
DeepLabCut/	Project registry and DLC projects; current jobs live within their respective DLC project.
Pipelines/	Saved Processing and Analysis pipeline definitions.
Logs/	Application and discovery diagnostics, where produced.

An installed main package is separate from this directory. It contains GUI/, Tools/, Processings/, Analyses/ and Spike_sorting/. Project scripts extend that installation; they should not be saved into GUI/ or Tools/.

When archiving or sharing, retain project settings, scripts, conditions, pipelines and relevant job outputs together with access to source NWB and external videos. Moving a source file or changing its drive letter can invalidate stored paths. Re-add moved files, remove obsolete inventory entries and review saved conditions. Do not assume automatic path remapping.

2. Files and filters

Add NWB recordings to the inventory



File-and-folder browser from the supplied guide. Add indexes the selection; it does not copy the source data.

1. Click Add NWB files in Files. Select individual .nwb files or select a folder to scan it recursively. Double-click a folder to browse inside it.

2. Wait for indexing. Workbench reads metadata, content paths and trial columns into Data/nwb_inventory.xlsx.

3. Select a known recording and inspect Content. Confirm that expected objects and trial/event information are present before running a large batch.

Adding an already indexed path refreshes that path. Adding a new folder retains unrelated entries. Overlapping selected folders are deduplicated by normalized path, but two copies of the same session at different paths are separate entries.

The inventory is a catalogue, not the recording itself. It does not watch directories continuously. If another program adds or replaces an NWB object, add that file again to refresh its catalogue. If a file was deleted from disk, a rescan does not necessarily remove its old row.

Check nwb_readable and read_error if a file cannot be opened. Indexing success is not a scientific validation of data conversion or synchronization.

Filter, then select

Action	Meaning
File filter	Limits the recordings visible in Files using indexed metadata.
Select a file	Makes that recording part of the current workspace selection.
Select a folder	Includes the represented descendant recordings, within the visible inventory.
Select all / Clear selection	Selects the displayed inventory scope, or clears its selection. These do not delete files.
Reset filters	Clears filter constraints. This is different from clearing selected files.
Delete in Files	Removes selected references from the project inventory; the physical NWB files remain on disk.

Combining constraints

Values chosen within one categorical field are alternatives (OR). Constraints in different fields are combined (AND). For example, session = morning or evening together with cohort = control means control recordings from either session. The actual fields come from your metadata; no genotype, species or task field is universally required.

Changing file filters rebuilds the visible file tree and can clear its selection. Select the intended files again and check the count. Changing only trial filters preserves the current file selection and does not narrow the displayed file list.

A practical example

1. Choose the metadata values describing the desired cohort or acquisition period.

2. Select all visible files, or Ctrl-click / Shift-click a smaller subset.

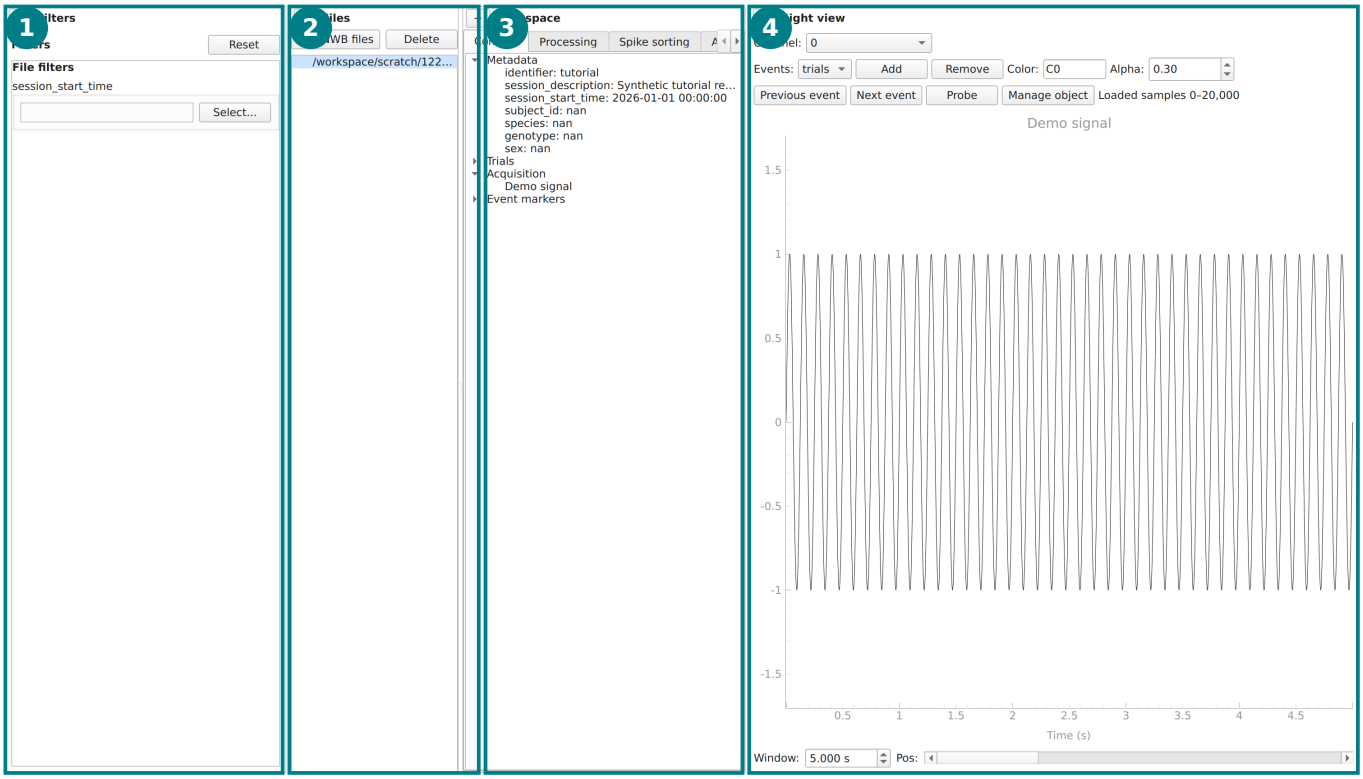
3. Inspect the Content summary to verify the required objects exist in those files.

4. Open the workflow and check its own eligible-file or selected-video count before running.

A visible recording is not automatically selected, and a selected recording is not automatically eligible for every tool. DeepLabCut additionally has a persistent video library with its own per-stage selections; Analysis has saved conditions that can retain a different file list.

3. Inspect your data

Read the four panels



Real application screenshot with a synthetic signal. 1: metadata filters. 2: Files. 3: Workspace/Content. 4: Right view. This earlier capture illustrates the panel layout; newer builds add more controls.

Panel	Use it to...
Filters	Restrict recording metadata and, in Analysis, eligible trial rows.
Files	Choose which recordings you are inspecting or passing to a workflow.
Workspace	Navigate Content or select a processing, DLC, sorting or analysis stage.
Right view	Inspect the chosen object or configure and run the chosen operation.

Drag the vertical dividers to resize panels. The small collapse control hides a panel to make room for a plot or video. Expand a Content branch and select an object rather than only its category header.

A multi-file or folder summary reports how many indexed files contain an object. That count is availability, not a statistical sample size or proof that all objects have equal duration, units or sampling rates. An object manager may open when an object belongs to several files.

Understand the objects in Content

Object or branch	What it represents	What to check
Metadata	Recording, subject and device descriptions	Session identity and consistent subject IDs.
Acquisition	Acquired signals or images	Timing, units, channel labels and origin.
Processing modules	Derived data grouped by purpose	Method, output name and provenance.
Stimulus	Stimulus descriptions or presentations	Timing relative to acquisition.
TimeSeries	Sampled values with timestamps or rate	Meaning of each dimension and unit.
ElectricalSeries	Electrophysiology linked to electrodes	Channel/electrode mapping and conversion.
ImageSeries / video	Images or references to video files	External paths and synchronized frame timestamps.
TimeIntervals / trials	Rows with start_time and stop_time plus metadata	What a row means, missing values and overlap.
Event-like objects	Time points or intervals	Whether onset, offset or both are meaningful.
Units	Per-unit spike times and associated metadata	Sorting label, curation provenance and time base.
Exported DLC body part	X, Y, original confidence and processing status	Associated video and project; pixels are not physical distance.

Names are chosen during conversion or processing. A label such as stimulus, response or fluorescence is not a reserved scientific meaning. Check the object type and metadata rather than inferring its contents from the name alone.

An unsupported extension may be indexed without having a specialized plot or extractor. Use a suitable script or extend the reader explicitly; changing its display name does not convert its data type.

Signals, events and synchronized views

1. Select a signal. Choose the channel deliberately; an Average view can hide channel differences.

2. Set Window and position to inspect a short interval, then move along the time axis by left-dragging the plot. Use the available zoom and position controls for precise navigation.

3. Choose an event marker and Add it to overlay its timestamps or intervals. Color and alpha affect display only. Previous event and Next event move to neighboring events.

4. Add compatible traces or a video to the shared view where offered. The linked views use the same NWB time axis, including during video playback. Two non-video traces can also be linked.

The shared time axis

NWB time in seconds and video elapsed time can differ because of recording offsets and synchronization. Linking displays does not estimate or repair synchronization. If one stream is shifted, check its stored timestamps and synchronization procedure before interpreting events across modalities.

For video, use Previous frame and Next frame, Play and replay speed, or enter an NWB time. Zoom with the mouse wheel, pan by left-dragging and fit with the Fit control or double-click. Frame count and elapsed-video time help identify the physical frame; the NWB timestamp locates it relative to other data.

Tracking overlays and exports

For exported DLC objects, use Show body part on associated video or the Body parts menu to add individual overlays. Filled circles indicate original positions, hollow circles interpolated positions and squares fixed estimates. Selecting a coordinate object plots X and Y as curves with separate vertical axes. Positions remain in pixels until an analysis explicitly normalizes them.

Right-click plotting options include grid, line width and Save displayed plot where supported. A saved plot is a visual export, not a numerical dump of the entire recording. Manage object can rename or delete NWB content, unlike Delete in Files, which only removes inventory references.

4. Events and trials

A common language for timing

An event marker tells Workbench when something occurred. It is a timing reference, not a data-selection rule by itself. A point event has one meaningful timestamp; an interval has a beginning and an end. A trials table is an interval table whose additional columns describe each observation.

Example	Interpretation	Useful reference
Stimulus onset	Instant of presentation	start
Stimulus presentation interval	From onset to removal	start or stop
Response timestamp	Instant of action or detection	start
Trial row	One task or observation interval	trials start / trials stop
Artifact interval	Period excluded or identified by a method	Interval boundaries

Examples are illustrative. The user who converted the data must know whether a timestamp means hardware trigger, exposure, software receipt, detection or another event. These can have different delays.

Do not confuse these operations

• Displaying an event adds a visual overlay; it does not filter trials.

• A trial filter chooses rows using metadata, such as outcome or stimulus category.

• Alignment cuts data into windows and expresses time relative to a chosen event.

• Processing may create or combine markers, but only when a specific tool explicitly does so.

Available marker choices are drawn from indexed objects across the analysis files. A marker appearing in the selector does not prove that it exists in every file. Missing boundaries are handled per file or segment and must be checked in the log.

Start, End and offsets

Setting	Example A: fixed window	Example B: variable window
Start marker / edge	stimulus / start	stimulus / start
Start offset	-0.5 s	-0.5 s
End marker / edge	stimulus / start	response / start
End offset	+2.0 s	+0.2 s
Time zero	Stimulus onset	Stimulus onset
Extracted interval	0.5 s before to 2 s after onset	0.5 s before onset to 0.2 s after response

The Start edge is time zero before applying its offset. An offset changes the extraction boundary, not time zero. Selecting trials/start to trials/stop with zero offsets extracts each complete trial.

Pairing rules in the standard Analysis extractor

• The same marker uses the same occurrence, including start and stop of one interval.

• For different markers, use the first End at or after Start, before the next Start. If Start belongs to a trial, that End must not exceed its trial stop.

• Pair the full event sequences first, then apply trial selection. Removing a trial must not change which End was paired with another Start.

• Missing End or an invalid resulting window causes a skipped segment. Offsets are applied after pairing and can extend beyond the enclosing trial.

• Windows include their beginning and exclude their end: [start, stop).

Standard continuous extraction copies regularly sampled values into relative-time bins without interpolation. Shorter windows and unavailable samples are NaN. Different rates or irregular timestamps require a compatible custom analysis rather than silent resampling. The corner-normalized trajectory script is an explicit exception: it performs controlled interpolation of irregular video samples.

Inspect event overlays on a known file before batching. For variable-duration averages, later time points may contain fewer trials or subjects; report these counts.

5. Processing

Create reusable derived data

Processing turns one or more selected NWB inputs into derived objects that can be inspected, reused and analyzed later. It is appropriate for filtering, transforming signals or constructing a new measure. It is different from Analysis, whose usual output is a figure and numerical result outside NWB.

1. Select the recordings in Files. Open Processing and choose a tool under its category.

2. Read the description and check Available files. Required input types and object paths determine eligibility; a selected file can be ineligible.

3. Choose each data input. A two-signal tool may ask for two separate objects; they must both be present and scientifically compatible.

4. Set parameters and output names. Use names that distinguish method variants and state units where relevant.

5. Choose whether existing outputs may be replaced. Run on one recording first.

6. Inspect the log, then inspect the new object in Content. Verify duration, sample count, units, channel structure and representative values.

7. Once validated, apply the same settings to more files or save a pipeline step.

What changes on disk

The standard file-tool runner opens each eligible NWB for writing, resolves selected inputs, calls the tool once for that file, adds returned objects to a processing module and updates inventory information. A custom tool can behave differently, so its description is part of the contract.

A saving_name parameter helps identify existing outputs before a run. With replacement disabled, existing outputs may be skipped rather than recalculated. With replacement enabled, verify the exact module and object names. Processing is not automatically restricted to the current plot window or the trial filters used for Analysis.

A failed file does not imply every other file failed. Conversely, a completed run does not mean every selected file produced a new result. Review eligible, completed, skipped and failed files before proceeding.

Plan a processing sequence

A generic example

Suppose each session contains a sampled sensor signal. You want to remove its baseline and then calculate a derived amplitude measure. First create a baseline-corrected object, inspect it, then select that object as the second tool’s input. Keep the original acquisition object for traceability.

Step	Input	Output / check
1. Inspect	Original signal	Timing, scaling and missing data understood.
2. Transform	Original signal	A new processing object with preserved or deliberately changed timing.
3. Validate	Derived object	Numerical values and metadata match the intended transformation.
4. Reuse	Validated derived object	Further processing or analysis.

Save as a pipeline

1. Configure the first tool and use Add to pipeline. Choose or create a named processing pipeline.

2. Add subsequent tools in dependency order. Confirm each uses the intended preceding output name.

3. Review saved parameters, targets and replacement choices. Editing current controls does not automatically edit a previously saved step.

4. Use the step controls to reorder, duplicate or remove steps. Save edits before launching.

5. Launch the selected steps and read their individual status and run report. If an upstream step failed, a downstream step could encounter an older object with the same name.

Processing and Analysis have separate pipeline collections. Definitions live in Pipelines/processing_pipelines.json and Pipelines/analysis_pipelines.json. They store workflow settings, not source data or Python dependencies.

Chapter 10 explains how to write a processing tool. A project-specific tool belongs in Script/Processing/<Category>/ inside the project. Built-in tools in the main package’s Processings/ are available to projects using that installation.

6. DeepLabCut

Project, model and environment

DeepLabCut is optional. The integrated workflow targets DLC 3 with the PyTorch engine and one animal per video. Do not assume that multi-animal identity tracking, 3D reconstruction or every DLC configuration is supported by this tab.

Term	Meaning
Workbench project	Your overall NWB study and its inventory.
DLC project	A tracking task with a config.yaml, body-part definitions, labeled frames and model files.
Named model	A training configuration/selection represented by a separate DLC shuffle.
Checkpoint	Weights saved during training. A trained model can have several checkpoints.
Analysis	Predictions obtained by applying trained weights to video frames. This is separate from the Workbench Analysis tab.

1. Enable DeepLabCut for the Workbench project, then open its tab.

2. Check the compact environment status. Green means detection/validation succeeded; red means a usable environment still needs attention.

3. Use the settings button next to that status for detection details, Browse and Test. Select the Python executable inside the DLC environment, not an arbitrary base Python.

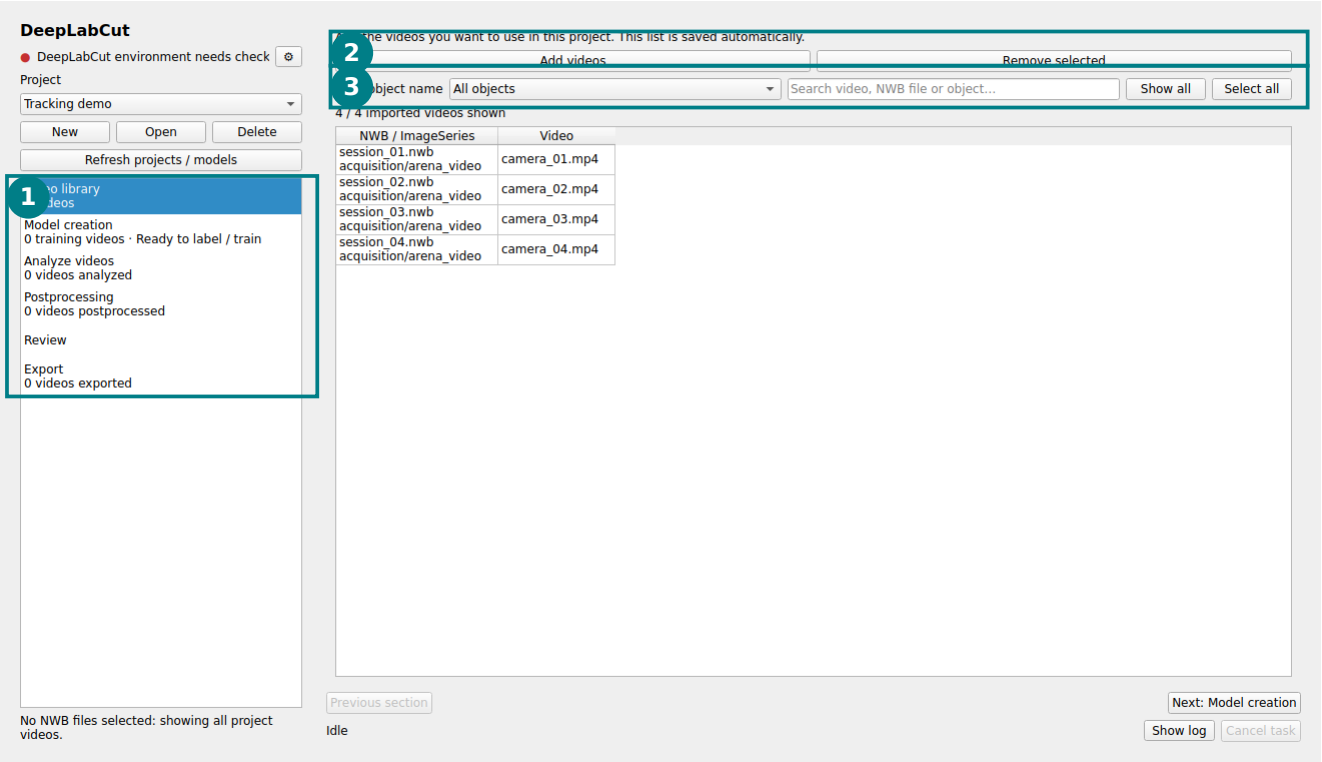
4. Create a DLC project with a name and body parts, or Open an existing config.yaml. Videos can be added after project creation.

5. Use the six stages: Video library, Model creation, Analyze videos, Postprocessing, Review, Export.

DLC jobs run in the selected environment; Workbench itself can use a separate environment. CUDA must be available to PyTorch inside the DLC environment. Having an NVIDIA GPU or driver alone does not prove that the installed torch build supports CUDA. The interface chooses an available GPU by default, with CPU selectable.

Delete project can remove a registry entry; permanent deletion of its folder, labels and models is a separate explicit choice in the dialog. Read that choice before confirming.

Import videos once into the library



Real current widget capture with four illustrative library entries. 1: workflow stages. 2: add/remove. 3: object-name and text filters. No training or inference was run for this example.

1. Select source NWB files in Files. In Video library, click Add videos to choose video references found in the indexed NWB content.

2. Filter by NWB object name when a file contains several cameras or video streams. Use the text search to narrow by video, file or object.

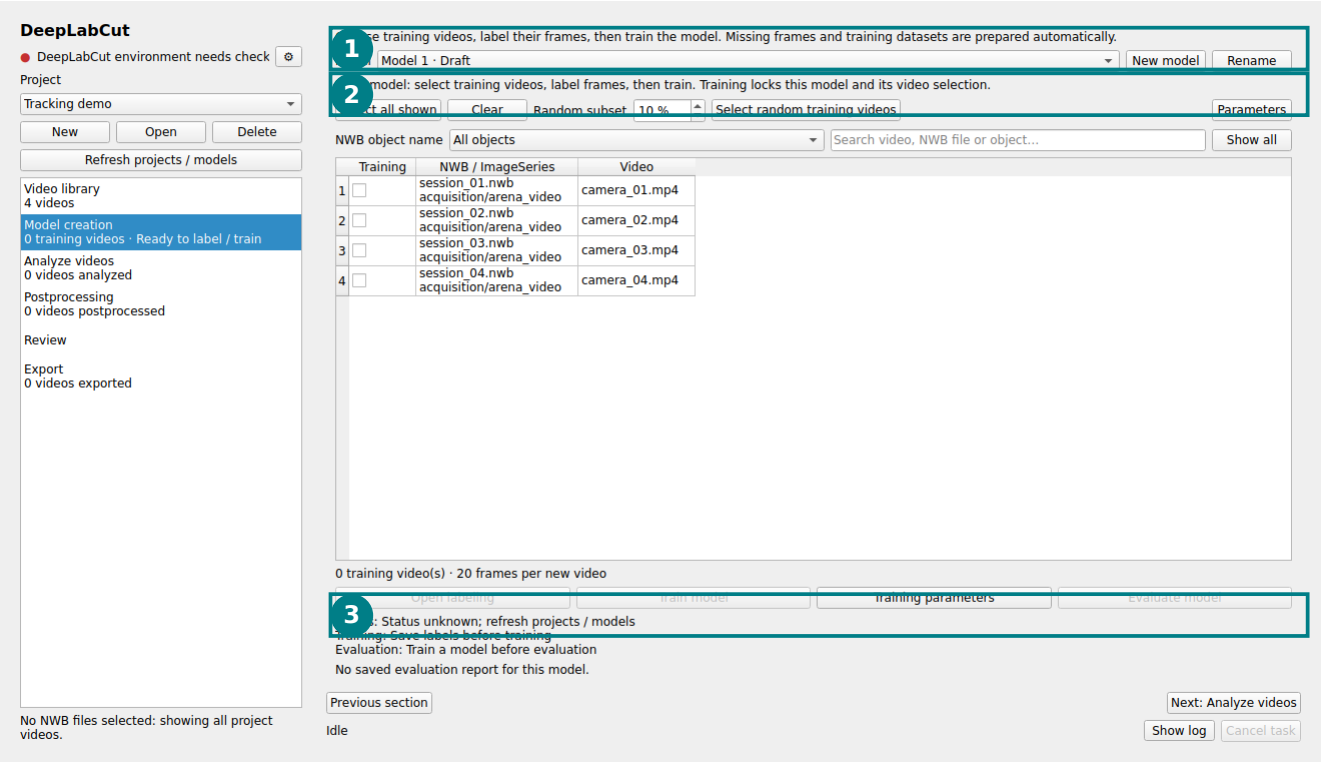
3. Keep the videos you need in the persistent project library. You do not have to rediscover the same videos every time you label or analyze.

4. To remove library entries, highlight rows and choose Remove selected. If labels or training history are affected, read the confirmation before continuing.

The library stores references. It is independent of the physical video storage choice used when registering training videos. In labeling parameters, Use originals uses links where supported; Copy videos creates project copies. Linked originals must remain accessible. Hard links share file data rather than duplicating its storage, and are not independent backups.

The visible library respects selected Files and name filters. With no Files selected, all project library videos are shown. Show all clears the limiting selection/filters, including the main Files selection. Hiding a row does not remove its saved library entry or necessarily clear its stage selection.

Model creation and training-video selection



Model creation in a demo project. 1: named model and New model/Rename. 2: selection controls. 3: labeling, training, parameters and evaluation actions. Disabled actions indicate unmet prerequisites.

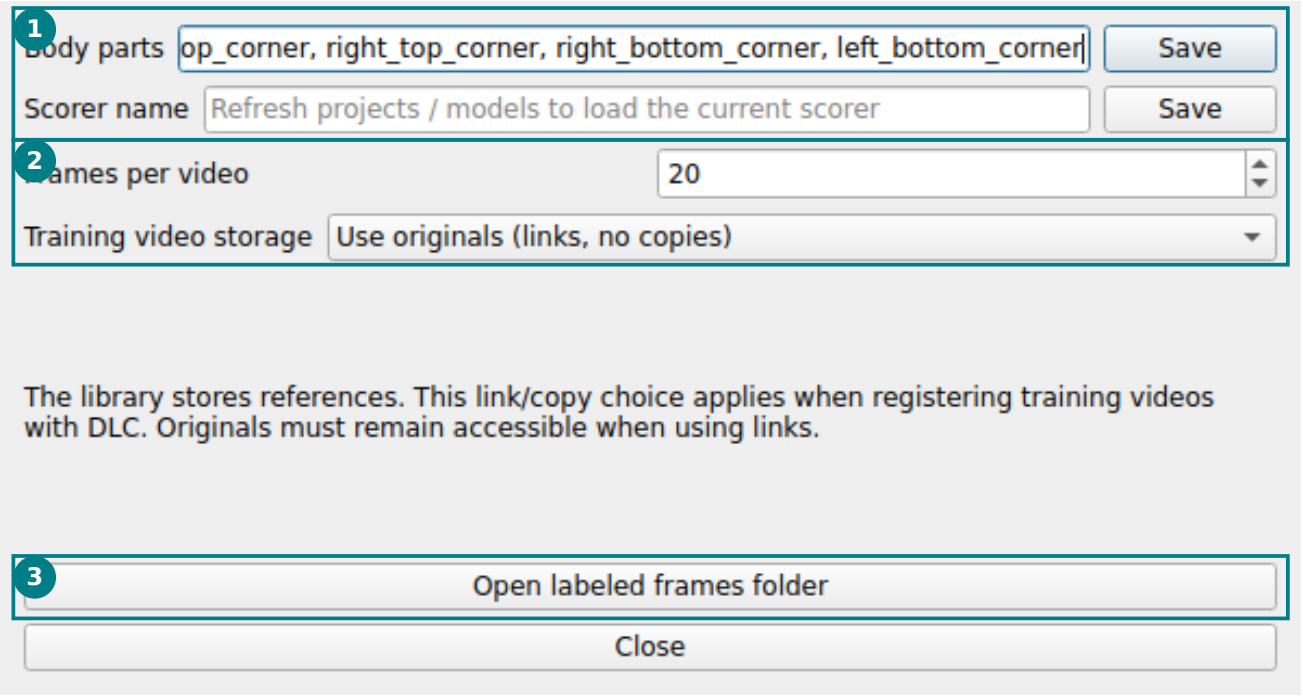
Create or choose a named model, then select representative training videos from the saved library. Select all shown, Clear and a random percentage reduce manual selection. A random subset is convenient, but inspect whether it covers the visual variability you expect: positions, illumination, backgrounds and recording sessions.

Training video selection is distinct from DLC's internal split of labeled frames into training and evaluation sets. Excluded/Test videos are kept out of future training selections, but an existing model may already have seen them. Calling a video "Test" does not retroactively make it independent.

Once a named model has been trained, Workbench locks its training-video selection and Train model action. Create a new named model to test another selection. Rename changes the human-readable identity; it does not retrain the network.

The Parameters dialog contains body parts, scorer name, frames per video and training-video storage. Scorer identifies the saved annotations and defaults to user. Set it deliberately before labeling. Append new body parts carefully; avoid renaming/reordering existing annotated parts. Label added points and train a new model when the definitions change.

Labeling parameters and frame selection



Real labeling-parameters dialog. 1: annotation definitions. 2: extraction count and storage. 3: open the extracted-frame folder. The displayed project is illustrative.

Setting	Choose it deliberately
Body parts	Consistent anatomical/landmark identifiers. Save changes before labeling. Changes to already annotated definitions need care.
Scorer name	The annotation identity, commonly user. This is not the trained model's prediction scorer string.
Frames per video	How many representative images to extract for labeling. More frames are useful only when labels are accurate and examples add meaningful coverage.
Training video storage	Use original linked videos or copies. This controls DLC registration; the Workbench library itself stores references.
Open labeled frames folder	Inspect extracted images and their annotation files. Avoid renaming folders manually while they are referenced by the project.

First select representative training videos in Model creation. A random subset is convenient but can miss rare behaviors or recording conditions. Check coverage across camera views, lighting, subjects and poses. A training-frame count is not a substitute for this review.

Open labeling handles extraction when needed and then opens Napari. Reopening an already labeled video is the normal way to inspect/correct its annotations. Saving labels does not automatically retrain a model or update predictions previously computed with it.

Label, train and evaluate

1. Click Open labeling. Workbench registers the selected training videos, extracts frames where needed and opens the Napari labeling workflow. Existing extracted frames and labels are retained.

2. Label the selected body parts consistently on each extracted image. Save the annotations in the labeling interface. Use the Workbench labeling navigator’s video list or previous/next controls to move between training videos.

3. Leave an occluded point unlabeled when its position cannot be determined reliably; do not invent a location just to complete a frame. Use a consistent anatomical definition.

4. Return to Model creation and check labeling status. If selected videos are not all labeled, Train model warns before continuing. Decide whether the labeled subset is adequate.

5. Open Training parameters. Check device and epochs. Click Train model; preparation of the training dataset is performed automatically when required.

6. After training, click Evaluate model. Inspect the train/test errors and prediction examples before using the network for a large analysis batch.

What creating the training dataset means

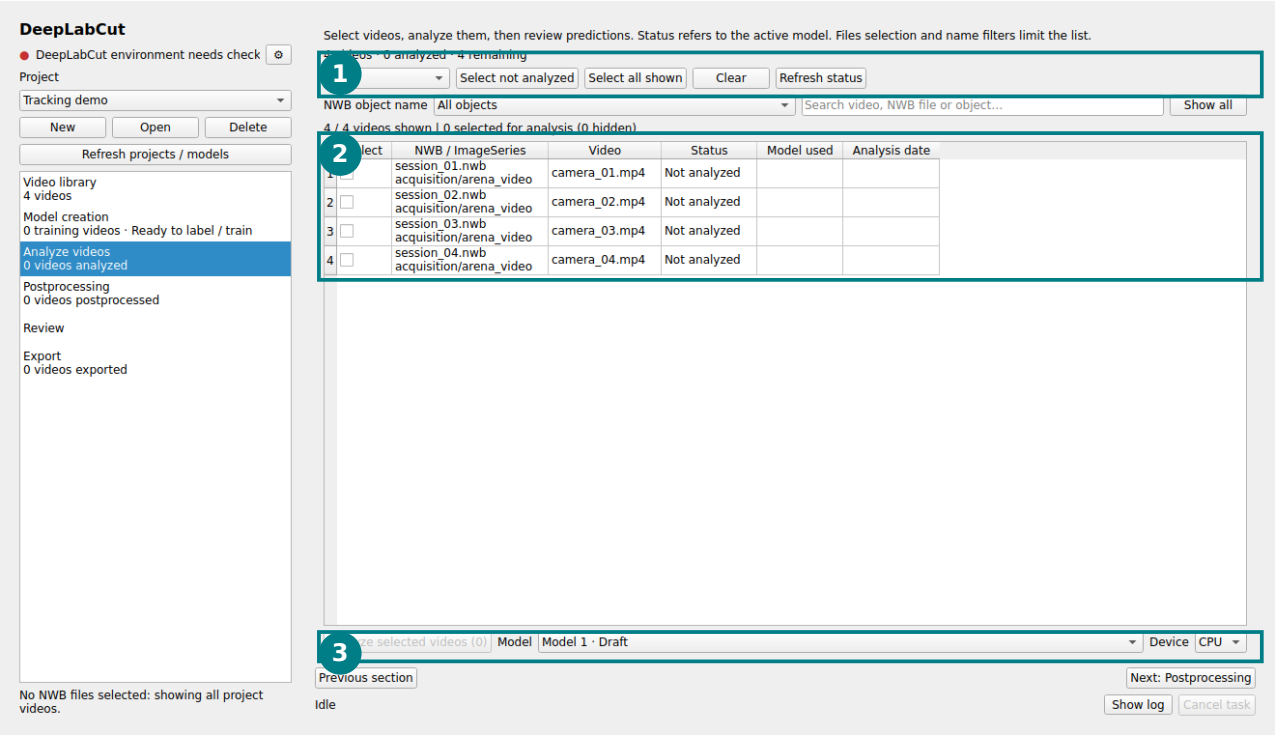
DLC collects saved image annotations, builds its training/evaluation split and writes the files needed by the training engine. This step prepares learning inputs; it does not itself fit the model and is not the same as importing videos into the library.

What evaluation can and cannot tell you

Evaluation compares predictions with labeled reference positions in the configured split. Interpret pixel errors relative to image resolution and landmark precision. A low training error alone is insufficient. Frames from the same session can be visually similar; a held-out frame score does not guarantee generalization to a new camera, session or subject.

Epochs is the requested training limit; DLC may stop earlier depending on its training configuration. Inspect the log and saved checkpoints. The tab’s simple training controls do not expose every advanced DLC configuration option.

Analyze videos and follow progress



Analysis selection, before a trained model is available in this demo. 1: status filter and selection tools. 2: per-video status/model/date. 3: Analyze, model and device.

1. Choose the trained model beside Analyze selected videos. Check the intended checkpoint/configuration when comparing results.

2. Select videos from the project library. Use Select not analyzed, Select all shown or Clear, and filter by status or video name.

3. Choose GPU or CPU for analysis. This choice is independent of the training device.

4. Click Analyze selected videos. Follow the current video in the log and the table, which updates after each completed file.

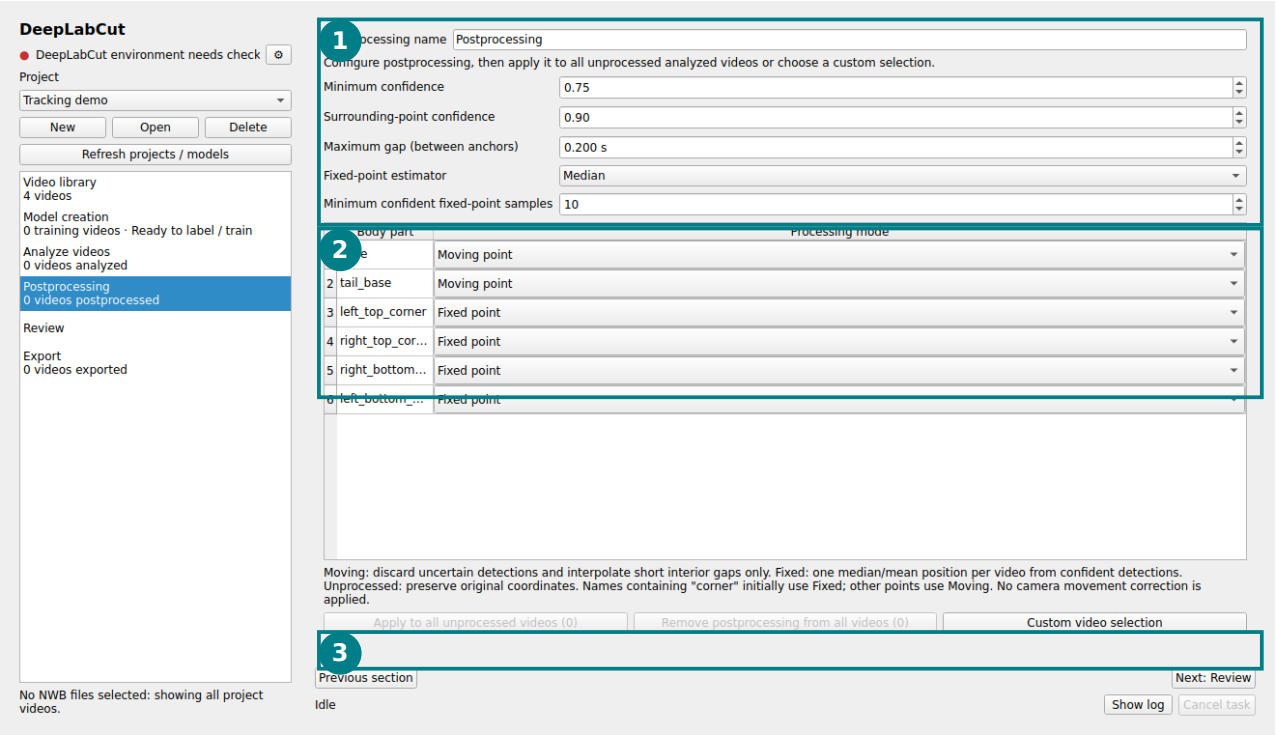
5. Review failed or incomplete videos separately. Reanalysis may be needed after source/model changes.

Completed video results are checkpointed during a batch. Canceling should retain previously completed videos; do not treat the currently interrupted video as finished. A completed file's prediction output is not yet an NWB export.

Current jobs and results live within the relevant DLC project under workbench/jobs/. They contain request/result information, predictions and logs used by later stages. Do not delete a jobs directory merely because the batch finished: Review, postprocessing and NWB export can still depend on it. Older projects may have legacy job paths.

A frame-count warning, decoder error or truncated video means the result needs investigation. Metadata frame counts alone do not guarantee every frame decoded. Verify the complete prediction-to-frame mapping before exporting.

Postprocess predictions explicitly



Current postprocessing controls. 1: name and thresholds. 2: per-part processing mode. 3: apply/remove for all or open a custom selection.

Mode	Operation
Unprocessed	Keep the original predicted coordinates.
Moving point	Discard uncertain positions and interpolate eligible short interior gaps.
Fixed point	Estimate one mean or median position for the complete video from confident detections.

Minimum confidence determines usable detections. Surrounding-point confidence is the stricter requirement for the two measured anchors around a moving-point gap. Maximum gap is the elapsed time between those anchors, using synchronized NWB timestamps. Long gaps and unsupported ends remain missing; the method does not extrapolate.

Fixed-point mode suits stationary corners or apparatus landmarks. Choose median or mean and a minimum confident-sample count. Too few usable detections leaves the point missing. Fixed mode does not compensate for camera movement.

Give the operation a useful name. Apply to all unprocessed videos uses available analyzed videos, or open Custom video selection and highlight rows with click/Ctrl/Shift. That selection is independent of Analysis and Export. Changing settings alone does not apply them. Remove asks for confirmation and deletes saved postprocessing directly, without archiving; raw predictions and existing NWB exports remain unchanged.

Review predictions

Review

Selecting a video loads its preview automatically. Filter analyzed videos by processed/unprocessed status and use Previous video / Next video. Settings exposes analysis-run choices. Review selection is independent of the Analyze videos checkboxes.

Dot size controls marker diameter. Show labels toggles tracking markers and the legend; Labels beside points adds names near markers. Processed switches between available processed and original positions. Minimum confidence changes the preview display, not stored raw predictions. Use wheel zoom, drag pan, Fit, frame stepping, replay speed and event navigation to inspect behavior.

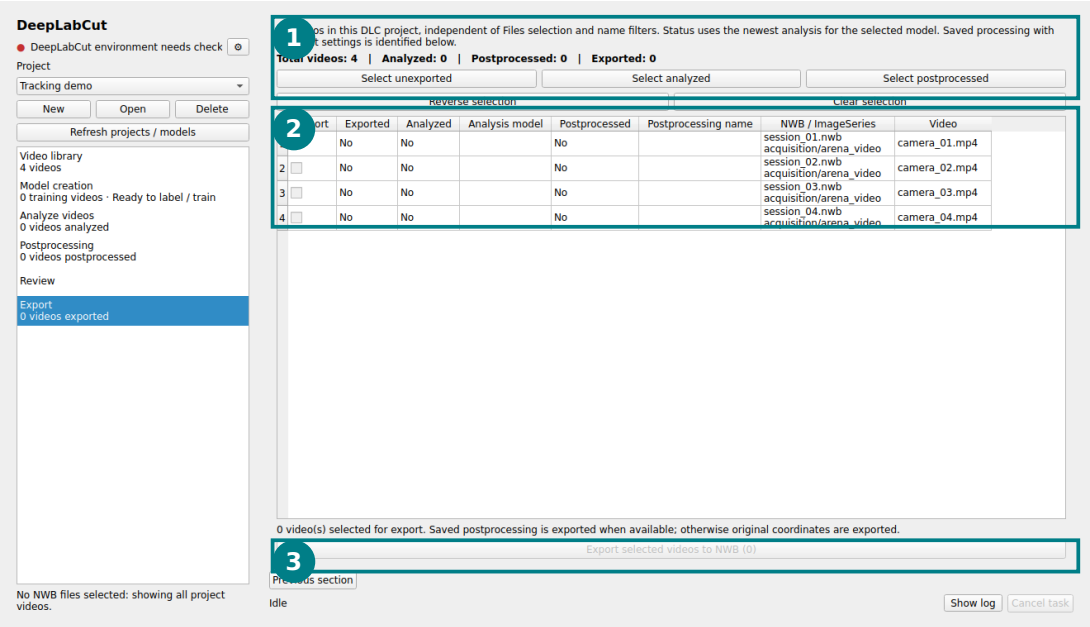
Control / marker	Meaning
Filled circle	Original predicted position.
Hollow circle	Interpolated moving-point position.
Square	Fixed-point estimate.
Processed	Compare saved postprocessing with original predictions.
Minimum confidence	Hide uncertain detections in the display; this is not a new processing run.
Dot size / labels	Adjust overlay readability without changing coordinates.
Previous / Next frame; Play; speed	Inspect short transitions or replay behavior.
Event marker and navigation	Locate frame times relative to stored NWB events.

A useful review pass

1. Inspect several intervals, including rare poses, occlusions and changes in lighting.
2. Zoom to check that points follow the intended body part rather than a nearby landmark.
3. Compare original and processed positions around each interpolated gap.
4. Check stationary points against the image before accepting a fixed estimate.
5. Use Export when the reviewed predictions are ready for downstream NWB analyses.

Review does not replace model evaluation on held-out labels. A visually plausible trajectory can still have systematic bias. Changing preview confidence or zoom does not repair a wrong frame-to-timestamp mapping.

Export tracking into NWB



Current Export stage with illustrative videos. 1: selection shortcuts. 2: export, analysis and postprocessing status columns. 3: export action and selected count.

1. Open Export. Its table covers project videos and indicates Analyzed, analysis model, Postprocessed, postprocessing name and Exported.

2. Use Select unexported, Select analyzed, Select postprocessed, Reverse selection or Clear selection as needed. Confirm the selected count on the export action.

3. Click export and read the summary of videos, models and applied processing. Choose whether to skip existing corresponding body parts or replace the existing tracking objects for the same source video.

4. Follow per-video status and logs. The table remains scrollable while export runs. Reopen an exported file in Content to verify the body-part objects.

If saved postprocessing exists for that result, export uses those coordinates. Otherwise it exports original coordinates. Export does not calculate unapplied postprocessing and does not write a second raw coordinate object alongside a processed one.

Column in each body-part object	Meaning
X, Y	Exported pixel coordinates, processed when available.
Confidence	Original DLC confidence, even for an interpolated/fixed position.
Postprocessing status	1 original, 2 interpolated, 3 fixed. Unprocessed exports use original. Missing coordinates remain NaN; original does not guarantee a valid detection.

One object is created per body part, named from body part and project. For example, nose_Tracking demo is the displayed NWB object name; its metadata bodypart remains nose. Preserve this distinction when entering names in scripts that read metadata.

7. Spike sorting

Select recordings and create a sorting project

Spike sorting is for extracellular recordings that retain spike-band information. A low-rate signal, an LFP-only output or an unrelated TimeSeries is not an appropriate substitute. The selected sorter must be compatible with the channel geometry and recording characteristics.

1. Select source recordings in Files, then open Spike sorting and click New.

2. Enter a Sorting name. Choose Data for sorting from the intended electrophysiology object, normally an ElectricalSeries with electrode information.

3. Decide whether to process each recording separately or concatenate groups. If concatenating, choose the file-filter metadata fields that define a group.

4. Review the resulting source-file lists and ordering. Confirm channel availability and the bad-channel policy.

5. Create the sorting project and inspect its summary before preprocessing or queueing a sorter.

What is stored

A sorting project under Data/Spike_sorting stores a recipe referencing source NWB files and the selected data object. It organizes preprocessing configurations, sorting runs, unit summaries, curation and exports. It does not replace the Workbench inventory or combine the original NWB files physically.

Eligibility is not scientific validation

Check sampling frequency, time units, electrode locations, channel IDs, scaling and bad-channel annotations. The application can only use metadata present in the NWB files. A sorter name in a dropdown does not mean its external dependencies, executable or GPU requirements are installed.

Run a small representative recording first. Inspect the preprocessed signal and the sorter’s output before a large queue. Default cutoff frequencies and sorter parameters are examples of a configuration, not a universal prescription for every recording.

How concatenation works

Concatenation presents several compatible recordings as consecutive chunks of one sorting input. It can let a sorter estimate units across recordings with a shared electrode configuration. It does not establish that an electrode samples the same neurons across days or that a unit identity is biologically stable.

Rule	Consequence
Grouping fields	Files with the same selected metadata values form a group. Use enough fields to separate incompatible recordings.
Ordering	Within groups, files are ordered by NWB recording/session start time where available, with deterministic fallback ordering. Inspect the actual list.
Missing grouping metadata	Missing values are handled separately instead of grouping all unknown recordings together.
Channel consistency	Concatenated chunks must use compatible channel identities and order.
Bad-channel mismatch	The dialog can retain only channels good in every file, or allow channels good in at least one file. The latter keeps problematic channels in some chunks.
Time mapping	Sorter frames are tracked back to source chunks so events and exported spikes can be placed in the original NWB time coordinates.

Example

Two sessions each contain 60 seconds of data but were recorded an hour apart. Their concatenated sorting input can have 120 seconds of samples. The hour-long wall-clock gap is not a minute-by-minute stretch of recorded samples. Do not interpret concatenated frame time as continuous session-clock time.

At export, a spike is assigned to its source chunk, converted from sample index with the appropriate sampling information and restored to that source’s timing. Preserve the recipe and timing map. Do not manually subtract guessed session lengths from exported spike times.

Grouping by subject alone is often insufficient when probes, channel maps or acquisition settings change. Prefer separate sorting projects/groups when compatibility cannot be established.

Preprocess and inspect the recording

1. Open the preprocessing stage for a recording/group. Use Preprocessing options to see the ordered operations and their parameters.

2. Enable or disable operations deliberately. The shipped examples include bandpass filtering and common-reference removal; available entries depend on installed/project scripts.

3. Check Channels and Probe to inspect the selected channels and geometry. Bad-channel handling and explicit channel selection affect every downstream step.

4. Inspect representative intervals, including artifacts and the boundaries of concatenated chunks.

5. Save the intended configuration before adding sorting runs. A different preprocessing sequence is a different input to the sorter, even when the source NWB object is unchanged.

Spike-sorting preprocessing usually builds a SpikeInterface recording transform chain. Operations can be lazy: the samples are computed when read or materialized. These settings do not by themselves create a new named signal in the source NWB Processing branch.

Sequence matters

Removing channels, filtering and referencing can produce different results when reordered. Referencing needs an appropriate channel grouping; avoid subtracting signals across unrelated groups. Filtering must respect sampling frequency and the target bandwidth. Inspect clipping, large transients and channel noise after applying the chain.

A different Python interface

A spike-preprocessing function receives a SpikeInterface recording and returns a recording. It is not a Processing TOOL receiving a PyNWB object. Project additions belong in Script/Spike_sorting/Preprocessing/. The PREPROCESSING declaration is documented in chapter 10.

A plot that looks smooth is not proof that the correct recording was selected. Confirm the data path, channel set and preprocessing settings stored with the run. Keep source data available for later analyzer and curation operations.

Configure sorting and unit summaries

1. Open the Sorting stage and choose the recording or group.

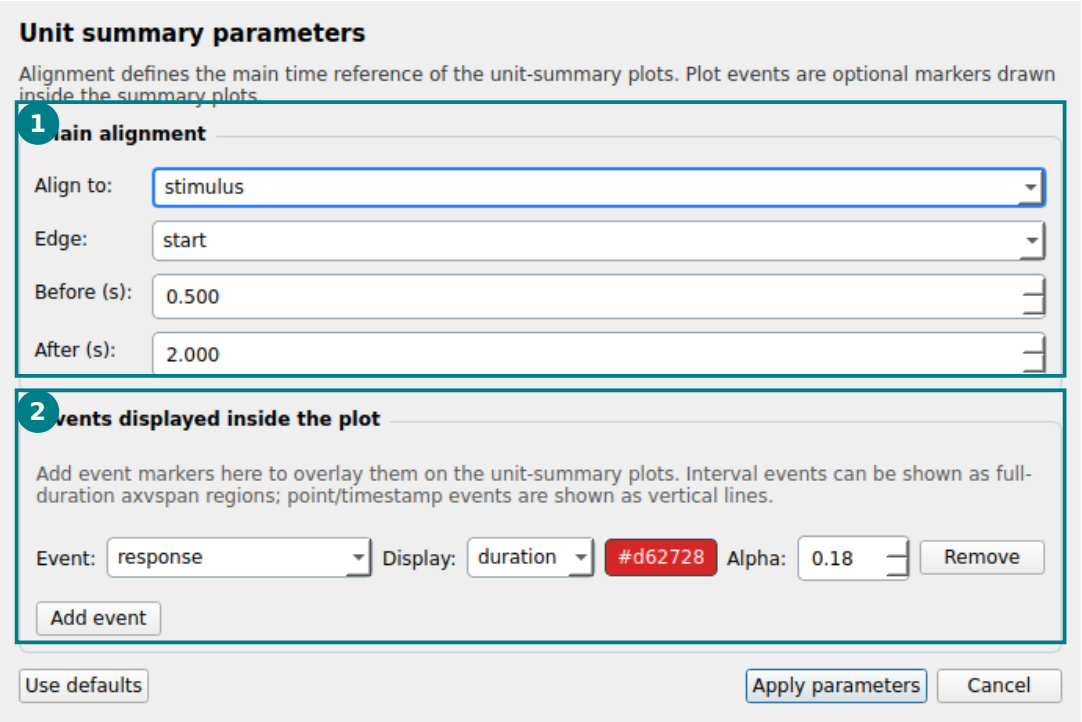
2. Choose a sorter and open Sorter parameters. Review parameters in the context of that sorter and your recording; defaults differ between backends.

3. Use Add configuration to queue a run. Give configurations descriptive names when comparing preprocessing or parameter choices.

4. Open Unit summary parameters before launching. Select the main event reference and the time window used in summary figures.

5. Choose whether a failed run stops the queue or whether Skip failed sorting allows later entries to continue.

6. Click Run sorting queue. Inspect each completed, failed or skipped configuration and its log.



Real unit-summary configuration dialog. 1: main alignment and window. 2: additional plotted events. These controls configure summary figures, not sorter detection parameters.

For summaries, the main event has start, stop or center options where supported, with Before and After durations. Additional plot events can appear as interval shading or timestamp lines, with color and alpha. This summary-specific interface is distinct from Analysis Start/End alignment.

Read the output figures

Unit summaries are inspection aids for each detected cluster. They are saved as PDF and PNG, and the browser offers previous/next unit navigation. Missing panels can reflect an unavailable analyzer extension or failed calculation; read the panel message and log.

Panel	What to inspect	What it does not prove
Autocorrelogram	Distribution of time differences between spikes, including behavior near zero lag.	An apparent refractory gap alone does not validate a single unit.
Waveform density	Shape and spread across waveforms/channels.	A compact shape does not rule out contamination or drift.
Maximum-channel waveforms	Individual waveforms and their median shape.	Amplitude alone does not identify a neuron.
Unit localization	Estimated spatial distribution relative to the probe.	Location estimates depend on geometry and signal quality.
Mean firing rate	Event-related changes in the chosen time window.	A response is not evidence of correct cluster separation.
Raster	Individual event/trial spike patterns and missing responses.	A mean response can conceal trial-to-trial heterogeneity.

Use the event-aligned plots to verify that timestamps and trial references make sense. For concatenated inputs, the mapping must preserve the source timing and avoid treating recording gaps as valid events. Summary timing is for inspection and does not modify the spike detector.

Configuring and comparing

Change Unit summary parameters to set the reference marker and overlays for generated summaries. Use identical references and windows when comparing runs. A summary parameter change can produce a different saved configuration even if sorter settings are unchanged. Do not compare plots with different time bases as though they were identical.

Assess unit quality using criteria appropriate to your experiment, including waveform stability, refractory violations, artifacts, cross-unit similarity and recording continuity. Workbench does not supply one universally correct acceptance threshold.

Manual curation and explicit export

1. Open Curation and select the intended sorting run. Confirm its preprocessing, sorter configuration and date.

2. Use Open curation to launch the supported SpikeInterface GUI or Phy workflow. Available choices depend on installed dependencies.

3. Inspect waveforms, spike trains, correlograms and available quality measures. Remove artifact/noise clusters; merge or split only when supported by evidence and by the selected editor.

4. Save curation in the external editor before closing it. When Workbench offers to apply the saved curation, confirm the correct source/run.

5. Check the resulting curated variant and its validation/details. Opening an editor is not the same as applying and saving its decisions.

6. Click Export, select run(s) and the intended curated or uncurated variant, then choose Excel and/or NWB outputs.

SpikeInterface GUI curation can be loaded from its saved curation_data.json. Workbench uses the available apply_curation support. A simple fallback handles kept/removed-unit information but cannot safely reproduce every complex edit. Phy export/import uses its saved sorting and cluster labels. Do not assume both editors interpret all quality labels identically.

Export option	Effect
Spike times as Excel	Tabular spike information, including source-file/time mapping.
Waveform templates as Excel	Template values and export metadata.
Create Units in source NWB files	Writes mapped spike trains and associated metadata to /units.
Units label	Stored as sorting_project; identifies a sorting set for later selection.
Replace same label	Replaces matching labeled units while preserving other labels.
Remove all existing NWB units	Replaces the entire table; broader than same-label replacement.

After export, inspect Units in each relevant source NWB. Verify labels, unit counts and representative spike times in the original time base. Keep the sorting recipe, original run, curation decisions and curated analyzer. Exporting to NWB does not preserve every external editor state by itself.

8. Analyses and conditions

Build a comparison deliberately

An analysis is a Python method with a GUI declaration. It can compare conditions, summarize signals, analyze events or read specialized NWB data. Choosing a script determines which inputs and parameters appear. Workbench does not impose the same statistical method on every dataset.

1. Open Analysis and choose a method in the Workspace. Read its description before selecting data.

2. Set the file filters and select the files for your first condition. In Analysis, trial-filter controls appear when the indexed dataset provides trial fields.

3. Set trial filters for the observations you want to retain. Add a condition, give it a meaningful name and color, then use Set current filter to capture this selection.

4. Repeat for other conditions. Inspect the saved file and filter counts under each row.

5. Use Save conditions to retain a named condition collection. Select current filter restores a condition’s settings for inspection or revision; Set current filter captures the revised state.

6. Select the method’s input data, alignment and parameters. Choose a result folder/name, run, and inspect the log and output.

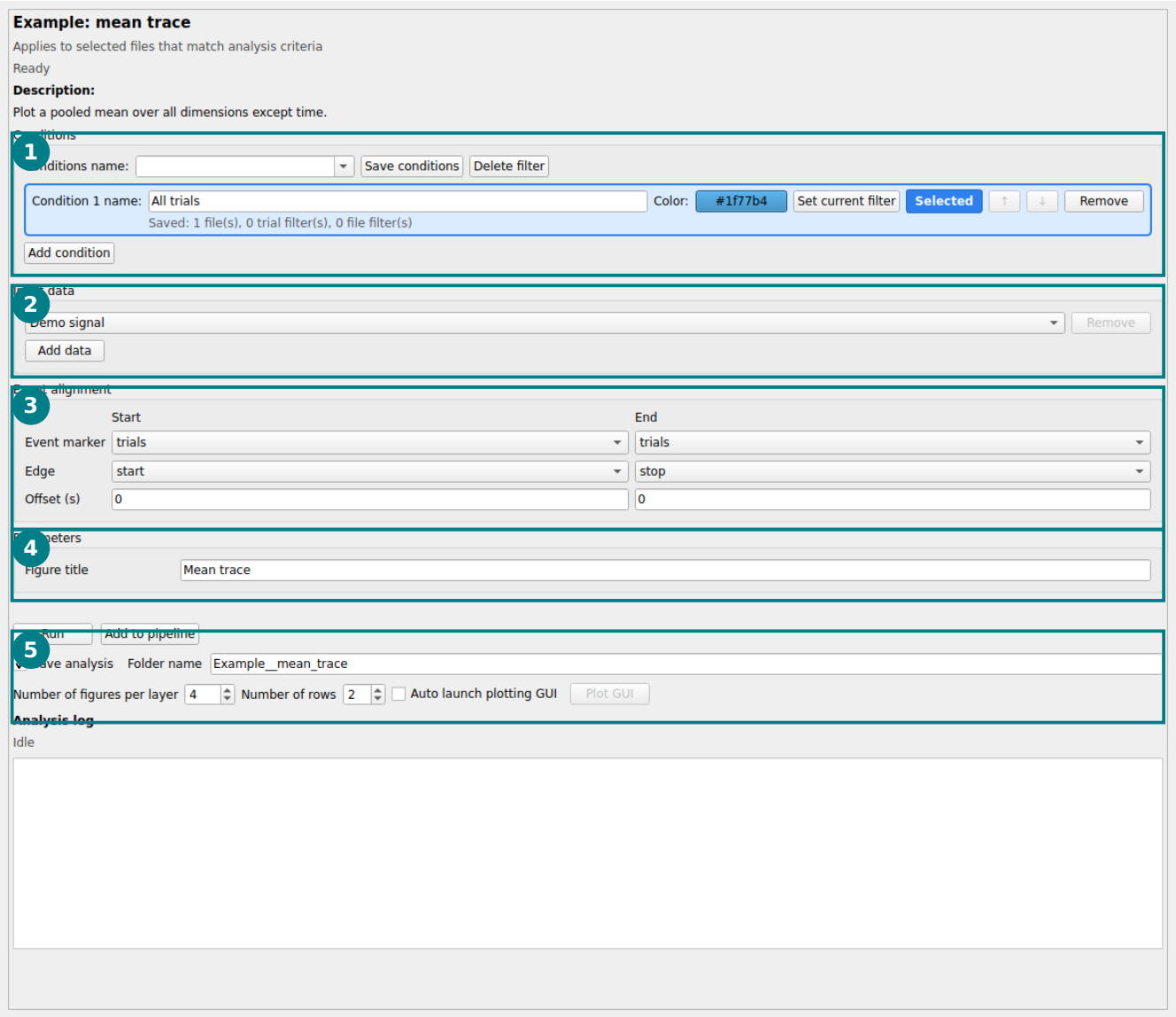
Conditions are saved selections

A condition combines the selected files with file and trial filtering criteria. It is not merely a legend label, and changing the current left-panel filters does not silently redefine every saved condition. Explicitly update the intended condition. Conditions can overlap; shared animals or trials are not independent observations just because they have different colors.

For example, compare successful and unsuccessful trials within the same recordings. Use the same file selection for both conditions, then capture different trial-outcome filters. For a between-group comparison, change the file selection or file filters as appropriate. Do not encode all experimental logic in a condition name while leaving identical criteria underneath.

Condition sets are stored in the project’s Filters area. Reusing a saved set improves consistency, but verify that paths, available trial fields and the intended data still match the current project.

Read the analysis form



Current form for the complete mean-trace example in chapter 11. A synthetic recording is selected; no experimental result is implied.

Area	Purpose
1. Conditions	Capture file and trial selection; name and color each comparison.
2. Input data	Choose the object(s) the method will receive.
3. Event alignment	Set Start and End references, edges and offsets.
4. Parameters	Method-specific choices. This example has only a figure title.
5. Run, saving and plots	Execute, retain output and open the figure organizer. The log follows below.

Understand the trial filters

Selection layer	What it restricts	Example
File filters	Which recordings are eligible/visible in Files.	Subject, group, session or protocol metadata.
File selection	Which eligible recordings you choose for the condition.	A subset of sessions from the visible tree.
Trial filters	Which rows of each selected NWB trials table qualify.	Outcome, stimulus category or numeric trial measurements.
Event alignment	Which time interval is extracted around each retained reference.	Trial start to trial stop, or an event-centered window.

Trial controls belong to the Analysis workflow, so they may appear when you enter that tab and disappear when you leave it. They depend on available trial columns. A project without a trials table cannot acquire trial-outcome filtering simply by adding a condition.

Categorical choices are combined with OR within one field and AND across different fields. Numeric bounds are inclusive. In the current ordinary numeric trial-filter implementation, missing values can be retained; a numeric range is not a guarantee that every selected row contains a measurement. If missing values must be excluded, make that policy explicit in the analysis. Trial-duration filtering has its own duration calculation.

A trial filter referring to an unavailable column can leave a file with no eligible trials. Review the log and actual observation counts, especially when a condition spans recordings converted with different schemas. The number of files in a condition is not the number of usable trials.

Example without assuming specific trial names

Suppose your conversion created an outcome column with values hit, miss and no_response. To compare hit versus miss, capture one condition with outcome=hit and another with outcome=miss. Use the exact stored values. Workbench cannot infer that “success”, “correct” and “hit” are equivalent across files.

Choose the statistical unit

Many trials from one subject do not create many independent subjects. Decide whether the script should average per trial, session or subject before comparing conditions. A plotting interface cannot correct an inappropriate unit of analysis. Keep that choice in the method description and saved output.

Select data and align events

The Input data area is controlled by the selected script. Add data can be enabled, disabled or unnecessary. Some methods accept one signal, some accept multiple signals or units, and some read the required objects directly from NWB. An empty Input data area is therefore not automatically an error.

What you see	Meaning / action
One input permitted	Select the required object. Adding several objects is not supported by this method’s declaration.
Several inputs permitted	Use Add data for each desired object. Confirm whether the method compares them, combines them or plots each separately.
No input required	Read the method description. A direct-NWB method may use body-part names, a video path or other parameters instead.
An object offered in the list	It exists in the indexed selection, not necessarily in every file. Check missing-object messages.

For sampled data, regular analysis preparation can extract intervals using Start and End event references. Each reference has an edge and an offset. A negative start offset includes data before the reference; an end offset extends or shortens the endpoint. Use the same marker’s start and stop to follow variable-length trials, or the same edge with different offsets for a fixed window.

Event names describe stored timestamps or intervals, not guaranteed biological meanings. Inspect them in Content first. If marker pairing or trial membership is ambiguous, resolve it in the method or conversion rather than assuming nearest timestamps are always correct.

What alignment does not do

Standard preparation does not promise to resample every signal to one shared sampling rate. Variable-length segments can contain padding/missing values, and coordinates describe valid durations. A method must handle these explicitly before filtering, FFTs, correlations or averages. Specialized direct-NWB methods can implement their own resampling and alignment rules.

The corner-normalized trajectory method is an example of a direct-NWB analysis. It looks up tracking and corners, applies the method’s timing/quality rules and performs its own alignment. Its body-part parameters refer to original DLC part identifiers, which can differ from exported object display names containing a project suffix. Read the method’s diagnostics when no trajectories qualify.

9. Results and the Plot GUI

Run, inspect and save

1. Check condition counts, required inputs, alignment and scientific parameters before launching.

2. Choose the analysis save options and a descriptive run/folder name. Use a new folder for a scientifically different configuration.

3. Run the analysis. Watch preparation, extraction and method messages separately; a preparation success does not guarantee usable data in the method.

4. Inspect the figures and saved tables. Compare trial/subject counts with your expectation, not just the visual appearance of curves.

5. Open Plot GUI when available to arrange and inspect registered Matplotlib figures.

6. Retain the result folder with its figures, tables and log. Record the application/script version used for a reported result.

Saved item	Purpose
Figure PDF (default format)	The registered analysis figures. Plot GUI can also save figures.
Excel tables	Pandas DataFrames associated with figures; multiple tables can be organized into sheets.
Pickle data	Non-DataFrame Python result objects when supplied. Only load trusted pickle files.
analysis_log.txt	Runner setup and execution messages, useful for tracing parameters and failures.
Project/filter/script files	The context needed to reproduce the run; a figure alone does not contain the full method.

By default, outputs are organized beneath Analysis in the project, using the analysis/run naming selected by the application. An explicit output directory can change this. Check the displayed path and log after saving. Reusing a folder and identical sanitized figure titles can overwrite files, so use distinct names for distinct runs.

Analysis outputs normally do not modify source NWB files. This differs from Processing and explicit DLC/spike-sorting exports. Saving a plot is not an NWB export, and a table in an analysis folder does not appear automatically in the Content tree.

Use the figure controls

The Plot GUI is a separate figure-management window. It arranges the figures registered by a method; it does not recompute the analysis when you change a display limit. Available plots and axes depend on what the script created.

Control	Use
Previous / Next	Move through the displayed figure groups.
Screen	Choose the display used for figure placement.
Figs / Layer and Rows	Control how figures are distributed in the layout.
Apply layout	Reposition figures according to the chosen arrangement.
Axis selector	Choose which plot axis receives limit changes.
X min / max; Y min / max	Enter display ranges, then Apply limits. Check units and avoid hiding relevant observations.
Save figures	Write the current figures. Choose a suitable location/name and inspect the saved files.
Close all	Close the displayed figure windows. Save desired changes first.

What a method must supply

Only figures registered through result.add are available to this workflow. A script that simply creates an unregistered figure or returns a dictionary has not necessarily supplied a saveable Workbench result. The developer example in chapter 11 shows the intended interface.

Logs and troubleshooting

Read the first specific error, not only the final “analysis failed” line. Typical causes are missing objects, zero retained trials, invalid event pairing, unsuitable dimensions or thresholds that leave no valid samples. “Data: none” can be normal for a direct-NWB method, but the method should then explain its own file exclusions.

When reporting a problem, include the method name, parameters, condition counts, relevant log messages and a small representative file or object description if sharing is permitted. A screenshot of the final exception alone often omits the actual cause.

Repeated workflows

Saved processing sequences and pipelines help repeat known steps. They do not make stale inputs or incompatible parameters valid. Inspect each stage’s status and result paths; only continue to an analysis when its required derived objects are present and current.

Prepared-data cache and reusable pipelines

Preprocessed data manager

Standard analyses can save filtered/extracted data for reuse in Data/Preprocessed_filtered_data. Here “preprocessed” refers to analysis preparation and alignment. It is distinct from a Processing tool that writes a derived signal into NWB and from the spike-sorting recording transforms.

Open Preprocessed data manager from Analysis to inspect cached entries, their conditions, source references, alignment, status and size. Open folder lets you inspect a selected location. Refresh updates the displayed cache inventory. Cached data can reduce repeated extraction work; they are not a substitute for saving the final analysis result.

Use its selection controls and Delete selected to remove unwanted cached entries/folders after checking the confirmation. Deletion removes this reusable extraction, not the source NWB recording or an already saved analysis figure. Recreating an extraction requires the original sources and compatible settings. Wait for active analyses/pipelines to finish before deleting cache data.

Analysis pipelines

1. Configure a method with its conditions, inputs, alignment, parameters and save options.

2. Use Add to pipeline, then choose or create a named analysis pipeline.

3. Inspect the stored step. Use Edit step and Save changes to revise it; changing the main form alone does not rewrite a saved step.

4. Order steps with the available move controls. Select the intended steps and Launch pipeline.

5. Review completed, failed and skipped status, logs and View run result for supported completed analysis steps.

Processing and Analysis keep separate pipeline definitions under Pipelines/processing_pipelines.json and Pipelines/analysis_pipelines.json. These definitions store workflow choices, not copies of every source recording or an installation of script dependencies. Retain the scripts and source paths they reference.

A cache or a completed pipeline does not certify the scientific method. After source changes or parameter revisions, verify that the run uses the intended extraction and that downstream analyses are not consuming an older derived object.

10. Write a processing script

Where scripts belong

Purpose	Main-package location	Project-specific location
Processing tools	Processings/<Category>/	Script/Processing/<Category>/
Analyses	Analyses/<Category>/	Script/Analysis/<Category>/
Spike preprocessing	Spike_sorting/Preprocessing/	Script/Spike_sorting/Preprocessing/

Main-package scripts are shared by projects using that installation. Put a method specific to one experiment in that project's Script directory. This keeps the scientific method with its project and avoids changing the shared default for everyone. Use .py files and descriptive categories and names.

Discovery imports Python modules and reads TOOL, ANALYSIS or PREPROCESSING declarations, or supported getter functions. Keep import-time code lightweight: do not open datasets, start a GUI or run a batch job at module import. The application must be able to import the script just to list it.

A project script with the same category and displayed name as a built-in method can override it. Use a unique name unless that replacement is intentional. Category folders organize the interface; the processing output module is a separate setting.

After adding or modifying a script, reopen/reload the project as supported, or restart Workbench to ensure the updated code is discovered. Check discovery/import logs if it is absent. Installing a script does not install its Python dependencies: those must exist in the Workbench execution environment.

Use the right interface

Processing transforms individual source files and usually writes derived NWB objects. Analysis consumes prepared condition data or explicitly reads NWB and creates scientific results. Spike preprocessing transforms a SpikeInterface recording. These are three different contracts, even if all three call a filtering function internally.

Inputs, outputs and the runner

For ordinary per-file work, use GUI.processing_runner.make_file_tool. It builds the TOOL entry point expected by the application. Your process_one function receives resolved parameters for one file. A raw TOOL func receives the transfer object, not a signal array; do not replace the helper with a direct signal function and expect identical behavior.

Input / context	Meaning
data with load_data=False	The selected PyNWB object. Read data, timestamps, rate, unit, conversion and offset as appropriate.
data with load_data=True	The stored numeric data array. Timing and physical-unit metadata are not carried as an object.
<parameter>_name	The selected object reference for that data parameter.
file_path / project_root	Source NWB location and project location.
inventory_row / nwbfile	Indexed metadata and the open NWB context supplied by the runner.
logger / tool_name	Logging and tool identity. Accept **kwargs for additional runner context.

Return a new supported NWB data-interface object, a list/tuple of such outputs, or a dictionary accepted by TimeSeries(**dictionary). Returning None creates no new object. An arbitrary DataFrame or free-form results dictionary is not a generic processing export contract. Do not both attach an object manually and return it for the runner to attach again.

The runner places outputs in a processing module, normally selected by params.category. Give each output a valid name and timing. A sample-by-channel array needs one timestamp per sample, or an appropriate starting_time and rate. Preserve or explicitly transform units and conversion metadata. If you convert values to physical units yourself, avoid applying the same conversion again on output.

Reading data[:] loads values into memory. Large recordings may need chunk-aware algorithms. Validate dimensions, sampling, missing values and required channels inside the function even if the GUI restricts the input type. A TimeSeries type check alone does not establish those scientific assumptions.

TOOL declaration reference

Field / helper argument	Purpose
name	Required displayed tool name. Keep it stable and distinctive.
description	Required explanation of input, transformation and output.
func	Raw entry point called with a transfer object. make_file_tool creates it for you.
params	Parameter definitions and runner settings; data fields become GUI selectors.
debug_mode	Run in debug mode rather than the ordinary background execution path. It can block the interface.
category	Discovery/display metadata. Do not confuse it with params.category, the output-module setting.
module	Discovery-supplied source module/path metadata. Normally set by the loader.
enabled_if	Optional callable receiving selected target paths; return whether this tool should be available.
project_action	Advanced project-level action instead of ordinary per-file processing; bypasses normal file eligibility and replacement UI. Its callable must implement that behavior.

Runner options and file eligibility

make_file_tool options

Argument	Purpose
process_one	Your per-file callable; required by the helper.
continue_on_error	Whether processing can continue to later files after an error. Review failures even if a batch finishes.
ui_title / ui_initial_text	Progress-window heading and initial message.
use_console_window	Optional console-style execution logging support.
params / debug_mode	Passed into the generated tool declaration.

Runner settings inside params

category specifies the output processing module. data_type_requirement lists required data references for file eligibility. match can restrict files using an inventory-column-to-predicate mapping, or a path predicate. All required predicates must pass; missing metadata can exclude a file. The helper supplies an NWB file-type constraint when needed. Advanced settings such as multiprocessing, tool_module and matlab_script_path are runner-specific, not ordinary scientific widgets; use them only after checking the relevant runner implementation.

internal_saving_name_list is an advanced output-name declaration used in conflict detection, including tools with several outputs. Ordinary saving_name fields populate the corresponding output-name list. Keep it consistent with the names the function actually returns.

A TOOL dictionary is executable Python configuration, not JSON. It can contain callables. Alternatively, a module can expose get_tool() returning its declaration. Do not place credentials, large arrays or open file handles in the declaration.

Parameter fields and selectors

Parameter type / field	Behavior
str, int, float, bool	Text, numeric or checkbox inputs. label controls the caption; default supplies the initial value.
list / choice	A choice widget; use values for list and choices for choice. These are alternatives, not arbitrary Python-list entry fields.
required	Marks a parameter that must be supplied. Still validate meaningful values in the function.
min / max	Numeric limits understood by the form. They do not replace algorithm checks.
data / nwb_timeseries	NWB object selector. neurodata_type_in limits offered types. load_data determines object versus array input.
saving_name	Output-name field used in conflict handling. Return an object using this name.
nwb_processing_module	Module selector, optionally allowing a new name with allow_new, new_default and new_label. The value is a reference/name, not an instantiated module.

```
"data": {
    "type": "data", "label": "Input signal",
    "required": True,
    "neurodata_type_in": ["TimeSeries", "ElectricalSeries"],
    "load_data": False,
},
"cutoff": {
    "type": "float", "label": "Cutoff (Hz)",
    "default": 30.0, "min": 0.0,
}
```

The dictionary key is the Python argument name. For example, cutoff becomes cutoff=... in your function. Labels are presentation only. An nwb_processing_module selector does not automatically replace params.category; your tool/runner configuration must use the selection deliberately.

File eligibility and output conflicts are separate concerns. A file can contain the required input but already contain the requested output. Inspect replacement/skip choices before running on important source data, and test a new tool on a small copy first.

Complete example: remove a temporal mean

Save the following function and the declaration on the next page together as Script/Processing/Examples/demean_signal.py. It supports one-dimensional or samples-by-channels data, preserves timing and returns a generic TimeSeries. It deliberately rejects per-channel conversion metadata rather than silently applying it incorrectly.

```
import numpy as np
from pynwb import TimeSeries
from GUI.processing_runner import make_file_tool


def demean_signal(data, saving_name, **kwargs):
    if data is None:
        raise ValueError("Select an input signal.")
    if not saving_name or "/" in saving_name:
        raise ValueError("Enter a valid output name.")
    values = np.asarray(data.data[:], dtype=float)
    if values.ndim not in (1, 2) or values.shape[0] == 0:
        raise ValueError("Expected samples or samples-by-channels.")
    if getattr(data, "channel_conversion", None) is not None:
        raise ValueError("Channel conversion is not supported.")
    values = values * float(data.conversion) + float(data.offset)
    values -= np.nanmean(values, axis=0, keepdims=True)
    if data.timestamps is not None:
        timing = {"timestamps": np.asarray(data.timestamps[:])}
    elif data.rate is not None:
        timing = {"rate": float(data.rate),
                  "starting_time": float(data.starting_time)}
    else:
        raise ValueError("No usable timing information.")
    print(f"Demeaned {data.name}: {values.shape}", flush=True)
    return TimeSeries(
        name=saving_name, data=values, unit=data.unit,
        description="Temporal mean removed from " + data.name,
        **timing,
    )
```

This is a teaching example, not a universal electrophysiology preprocessing method. It does not preserve an ElectricalSeries electrode association, does not remove artifacts and does not solve missing-value policy. An all-missing channel remains missing.

Complete the declaration and test it

```
TOOL = make_file_tool(
    name="Example: demean signal",
    description="Remove each channel's temporal mean.",
    process_one=demean_signal,
    continue_on_error=False,
    debug_mode=False,
    params={
        "category": "Examples",
        "data": {
            "type": "data", "label": "Input signal",
            "required": True,
            "neurodata_type_in": ["TimeSeries", "ElectricalSeries"],
            "load_data": False,
        },
        "saving_name": {
            "type": "saving_name", "label": "Output name",
            "default": "demeaned_signal", "required": True,
        },
    },
)
```

1. Create a tiny test TimeSeries with a known offset and known timestamps or rate.

2. Call demean_signal directly and check its values, shape, unit and timing.

3. Discover the script in a test project and run it on one copied NWB file.

4. Reopen the NWB and inspect processing/Examples/demeaned_signal in Content.

5. Test a duplicate output name, missing input, multiple channels and invalid input before running a batch.

Do not estimate sampling rate from the number of samples alone. Use stored timing, and verify that irregular timestamps are compatible with your algorithm. Do not mutate the source object’s array as a side effect when the intended output is a new derived signal.

Spike-preprocessing extension

A spike-preprocessing script uses PREPROCESSING. It receives a SpikeInterface recording and returns a recording. Save a project-specific file in Script/Spike_sorting/Preprocessing/. It will appear among the sorting workflow’s operations, not as an ordinary Processing tool.

```
import spikeinterface.preprocessing as spre


def bandpass(recording, freq_min=300.0, freq_max=6000.0):
    nyquist = recording.get_sampling_frequency() / 2
    if not 0 < freq_min < freq_max < nyquist:
        raise ValueError("Require 0 < low < high < Nyquist.")
    return spre.bandpass_filter(
        recording, freq_min=freq_min, freq_max=freq_max
    )


PREPROCESSING = {
    "name": "Project bandpass",
    "description": "Bandpass before spike sorting.",
    "func": bandpass,
    "default": False,
    "params": {
        "freq_min": {"type": "float", "default": 300.0},
        "freq_max": {"type": "float", "default": 6000.0},
    },
}
```

name and description explain the operation; func is the transform; params supplies its adjustable arguments; default controls its default activation. The frequencies above are illustrative and may be unsuitable for your recording. Preserve channel identities and timing unless the operation explicitly handles the consequences.

A lazy recording transform may not read every sample immediately. Test get_traces on representative chunks, and then test the configured chain with the intended sorter. Returning a NumPy array instead of a recording breaks this interface.

11. Write an analysis script

The standard function contract

```
def my_analysis(data_manager, result, **kwargs):
    # Read prepared conditions; create figures and tables.
    ...
```

The runner calls the declared function with data_manager, result and parameter/context keywords. Returning a dictionary is not the result-saving contract: register outputs with result.add. Use **kwargs if your function does not consume all runner context.

Manager method	What it provides
get_data_types()	Keys for the selected/prepared inputs. These may be display identifiers rather than raw NWB paths.
get_conditions()	Prepared condition names.
get_data(data_type, condition)	Prepared data for one input and condition. Signal-like data are often xarray objects; trials can be DataFrames.
get_data_nwb_path(data_type)	Underlying NWB reference information when available.
get_color(condition)	The condition’s configured color.
get_trials_df()	Combined trial information when trials were prepared as an input.

Inspect type, dims, coords, shape and units before assuming an array layout. Dimensions can include time, channels, trial/event, subject and unit identifiers depending on the selected extractor. Spike data can contain variable-length event arrays; they are not necessarily sampled binary traces.

Additional runner context includes selected_files, analysis_name, project_path, palette, save_analysis, analysis_save_name, analysis_output_dir, align_to_event, align_to_event_name and event_alignment. Avoid reusing these names for unrelated UI parameters. The manager is the standard source of filtered data; selected_files alone does not encode every prepared observation.

Alignment, missing values and aggregation

Prepared aligned data can include reference_time, end_event_time, segment_start, segment_stop, segment_duration_s, end_event_relative_s, segment_stop_relative_s, valid_sample_count and sample_grid_offset_s coordinates. Their presence depends on the extraction route. Use them to distinguish valid observations from padding and to understand each segment's time origin.

The event/trial dimension can be named after the marker, so a hard-coded dimension called trial is not always correct. Print a small shape/coordinate summary while developing. Validate the actual required dimensions instead of averaging every axis accidentally.

Preserve scientific meaning

• Specify whether pooling is across samples, trials, recordings or subjects. Unequal trial counts can give some subjects more weight.

• Use an explicit NaN and missing-data policy. Do not turn padded samples into zeros before estimating a mean or spectrum.

• Resample only when justified, with a stated target grid and maximum interpolation gap. Do not interpolate across arbitrary missing intervals.

• Keep each modality's units and timing. A visual time-axis match does not prove synchronization.

• Report counts of accepted and rejected observations and the reasons for exclusions.

Result registration and logging

```
result.add(fig, data=summary_dataframe)
print("Accepted 42 trials from 6 recordings", flush=True)
result.add_log("Summary complete", accepted_trials=42)
```

result.add associates data with a figure. Passing fig=None does not provide a general standalone-table registration route in the current implementation. result.reset_figures clears registered figures and their associated data; use it only when replacing results intentionally.

result.add_log retains structured entries, but do not rely on it alone for live GUI diagnostics. Use print(..., flush=True) for messages that should be visible during execution and captured by the runner log. Let the runner manage the normal save workflow rather than saving repeatedly inside a loop without a reason.

ANALYSIS declaration reference

Field	Purpose
name	Required unique displayed method name.
description	Required explanation of inputs, assumptions and outputs.
func	Required callable receiving data_manager and result.
params	Scientific parameter widgets; supported basic types include str, int, float, bool, list and choice.
input_spec	Defines accepted data, multiplicity and whether manual input selection is required.
align_to_event_marker	False/None: no required alignment; True: require alignment; dictionary: configure/fix alignment controls.
debug_mode	Debug execution can run on the GUI thread and block interaction. Normally False.
enabled_if	Optional callable of target_paths that determines whether the method is available. Keep it lightweight.
input_mode	Use raw_nwb only for a method implementing the direct-NWB contract. Omit for standard prepared data.
category / module	Discovery metadata used to organize and locate the method.

A module can expose ANALYSIS directly or get_analysis() returning the declaration. Parameters use label, default, required and applicable numeric min/max. A list uses values; a choice uses choices. In an analysis, a data-like parameter may be only a reference string, not the resolved PyNWB object passed to a Processing function. Prefer input_spec for standard prepared inputs.

A good description tells users what one data selection means, whether multiple objects are combined or processed separately, what trials/events are used, what the averaging unit is and which assumptions can invalidate the result. Do not make users infer these from a plot title.

Input and alignment specifications

input_spec field	Meaning
accepted_data	Allowed NWB type names, for example TimeSeries or ElectricalSeries.
accepted_names	Optional restriction to object names. Use only when name-specific semantics are intentional.
allow_multiple	Whether users can add several inputs. Default is False.
required	Whether an input must be selected. Default is False.
mandatory_data	Required named inputs resolved by the application; missing required data must be handled as an error.

```
"input_spec": {
    "accepted_data": ["TimeSeries", "ElectricalSeries"],
    "allow_multiple": True,
    "required": True,
},
"align_to_event_marker": True,
```

Advanced alignment dictionary

The supported keys include required, event_marker, event_path, edge, end_event_marker, end_event_path, end_edge, start_offset_s and end_offset_s. Dictionary values can preset or fix the corresponding controls. An omitted end reference can use the start reference. Legacy pad_before_s maps to a negative start offset and pad_after_s to the end offset; prefer the explicit offset fields in new scripts.

Use a Boolean True if the user should choose alignment freely. Do not hard-code a marker called stimulus unless that is a real requirement of the method. Edge semantics and offsets should be clear in the description. The application prepares intervals; your method must still check that they are suitable for its computation.

Allowing several data inputs is only a UI permission. Your function must iterate over or combine them deliberately. A declaration that accepts all TimeSeries should not then assume every input has voltage units or a channel axis.

Complete example: plot a pooled mean

Save this function and the declaration on the next page as Script/Analysis/Examples/mean_trace.py. It plots one figure per selected input and one curve per condition. The example pools all dimensions except time; it is not a subject-level statistical estimator.

```
import pandas as pd
import matplotlib.pyplot as plt


def mean_trace(data_manager, result, title="Mean trace", **kwargs):
    for key in data_manager.get_data_types():
        fig, ax = plt.subplots()
        tables = []
        for condition in data_manager.get_conditions():
            data = data_manager.get_data(key, condition)
            if "time" not in getattr(data, "dims", ()):
                raise ValueError("Expected an xarray time dimension.")
            dims = [d for d in data.dims if d != "time"]
            trace = data.mean(dim=dims, skipna=True) if dims else data
            if trace.size == 0:
                raise ValueError("No samples in this condition.")
            ax.plot(trace.time.values, trace.values,
                    label=str(condition))
            tables.append(pd.DataFrame({
                "condition": str(condition),
                "time_s": trace.time.values,
                "mean_value": trace.values,
            }))
        if not tables:
            plt.close(fig)
            raise ValueError("No conditions were provided.")
        ax.set(title=f"{title}: {key}", xlabel="Time (s)",
               ylabel="Mean signal (input units)")
        ax.legend()
        fig.tight_layout()
        result.add(fig, data=pd.concat(tables, ignore_index=True))
        print(f"Created mean trace: {key}", flush=True)
```

Before applying this template scientifically, choose the correct averaging dimensions and weighting. Inspect whether time coordinates are compatible, and report valid sample counts. The example accepts existing prepared timing; it does not invent a common resampling grid.

Complete the declaration and validate

```
ANALYSIS = {
    "name": "Example: mean trace",
    "description": "Pooled mean over all dimensions except time.",
    "func": mean_trace,
    "align_to_event_marker": True,
    "debug_mode": False,
    "params": {
        "title": {
            "type": "str", "label": "Figure title",
            "default": "Mean trace",
        },
    },
    "input_spec": {
        "accepted_data": ["TimeSeries", "ElectricalSeries"],
        "allow_multiple": True,
        "required": True,
    },
}
```

1. Test the function with a tiny manager containing known xarray data and two conditions.

2. Verify numerical means, not only that a figure opens. Include missing values and unequal trial counts.

3. Discover it in a small project, capture conditions and choose a known signal.

4. Check event alignment against the Content viewer and manually inspect a few trials.

5. Save results and reopen the figure, table and log. Confirm that the table contains the numbers plotted.

A successful run is not evidence of a scientifically valid method. For inferential analyses, add tests for the statistical assumptions and experimental design. Document the version and dependencies alongside your project-specific script.

Advanced: read NWB directly

Use input_mode="raw_nwb" only when standard prepared data cannot represent the method adequately, for example an irregular tracking workflow requiring its original timestamps and metadata. This route is implemented in the current runner but may be absent from older installations.

Here data_manager is a lightweight context with condition_selection and selected_files. It is not the standard filtered manager and does not provide get_data. The script must open NWB files read-only, resolve objects, apply condition/trial criteria, perform event alignment, handle missing data and register results. Merely setting input_mode does not perform those steps for you.

```
ANALYSIS = {
    "name": "Project direct-NWB method",
    "description": "Method implements its own NWB selection/alignment.",
    "func": direct_nwb_analysis,  # Your implemented function.
    "input_mode": "raw_nwb",
    "input_spec": {"required": False},
    "align_to_event_marker": True,
    "params": {},
}
```

The fragment above is a declaration pattern, not a complete analysis. Use the existing direct-NWB trajectory implementation as a reference for this runner version. Keep file handles in context managers and copy required values before closing them. Do not use this route merely to bypass a missing-input error without implementing the selection semantics.

Make failures explainable

For each rejected file, report the missing object, unavailable event, invalid corner geometry, insufficient confidence or other specific reason. Then report accepted files/trials. A single final “No usable data” exception is insufficient for a user to correct the setup. Distinguish an absent object from an object present but rejected by thresholds.

Keep direct-read methods read-only unless they are explicitly designed and presented as data-modification tools. A user launching an analysis should not unexpectedly have their source NWB files rewritten.

12. Practical checks and troubleshooting

Resolve common problems

Symptom	Check first
No files or unexpected files	Active file filters, current selection, missing paths and inventory state. Resetting a filter and selecting all are different actions.
Data listed but not usable	The object may not exist in every selected file, or its data/timing may fail the method’s requirements.
No events or zero trials	Marker path, edge, trial columns, saved condition criteria and selected time window.
Video unavailable or wrong timing	External path, decoded frame count, NWB timestamps and camera synchronization. Playback rate does not repair synchronization.
DLC cannot launch	Detected environment details and import/dependency log. Do not mix Workbench and DLC environments blindly.
CUDA not offered	Device detection in the relevant environment. A GPU driver alone does not make a CPU-only PyTorch build CUDA-capable.
Analysis stops reading a video	Check decoding and storage access. Do not treat a partial prediction file as a complete result.
Processing/script absent	Location, declaration fields, imports, name/category collisions and discovery logs.
NWB export missing in Content	Export success, target source file, chosen replacement policy and inventory/content refresh.
Empty analysis result	Per-file exclusions, selected input names, confidence thresholds, valid samples and alignment.

Keep source recordings and project outputs on storage accessible throughout a job. Network interruptions can affect both reading and result writing. Before restarting an interrupted long job, inspect which items are marked complete and whether their output files are present and valid.

What to keep, and a first-run checklist

Keep the context with the result

Retain source NWB files and external video references, the Workbench project configuration/inventory, project scripts, saved condition definitions, and the relevant DLC or sorting project with model/run metadata. Keep exported results and logs together. Avoid deleting job folders merely because a prediction or figure is visible; first establish where the durable outputs referenced by that project are stored.

A small first run

1. Create a project with two or three representative NWB files.
2. Inspect one signal or video and its events in Content. Verify units, duration and timing.
3. Apply a file filter, select a subset, and confirm which files are actually selected.
4. If needed, run one processing tool on a copy and inspect its output object.
5. Capture two analysis conditions and check file/trial counts.
6. Run a simple analysis and reopen its saved figure, data and log.
7. Only then expand to a large batch or an optional DLC/spike-sorting workflow.

Scope of the documentation

This revision was checked against the supplied Workbench source, especially project/script discovery, file and filter panels, processing and analysis runners, DLC job/export controls, spike-sorting workflow and result/plotting utilities. Screenshots show real widgets; illustrated library/model entries are examples. The older general-view captures are identified in their captions. Layout can vary with screen size, theme and installed version.

This guide describes application behavior and interfaces, not validation of a particular scientific analysis, camera synchronization, neural-unit classification or trained model. The complete example scripts illustrate the contracts; adapt and validate them for your own measurements.

If a control differs in your installed build, compare the application version and source before changing a dataset to fit the screenshot. When asking for support, include the relevant log, method name, parameter values and minimal reproducible data description.